

AppSpeX — Observability & Quality Improvement Plan

This document is based on evaluation results from multiple prompts:

- CRM (strong output)
- E-commerce (significant degradation)
- Marketplace B2B/B2C (partial degradation)
- Spaced Repetition / RetainHQ (business-rule degradation)

The goal is not to make the AI appear smarter.

The goal is to make the system transparent about:

- What it understood
 - What it generated
 - What it repaired
 - What it missed
 - How trustworthy the result is
-

Core Thesis

The biggest lesson from evaluation is:

AppSpeX succeeds on simple domains and degrades as requirements become more complex.

The answer is not:

- More styling
- More animations
- More visual polish

The answer is:

Observability.

A reviewer should immediately understand:

1. What the model extracted.
2. What assumptions were made.
3. What requirements were modeled.
4. What requirements were missed.

5. Which repairs occurred.
6. Whether generated workflows are original or synthesized.

The system should be honest about its limitations.

Findings From Evaluation

CRM Prompt

Result: Strong

Generated:

- Correct entities
- Correct pages
- Correct APIs
- Correct integration
- Correct workflow

No repairs required.

This proves the pipeline is capable of producing high-quality outputs.

E-commerce Prompt

Result: Weak

Prompt included:

- Products
- Orders
- Customers
- Stripe payments
- Gmail confirmations

Issues:

- Payment entity missing
- Integration hooks missing
- Pages missing
- APIs missing
- Workflows synthesized by fallback

Output quality degraded significantly.

Marketplace Prompt

Result: Partial

Prompt included:

- Login
- Payments
- Roles
- Chat
- File uploads
- Analytics
- Marketplace

Issues:

- Chat missing
- Analytics missing
- Payments missing
- File uploads missing

The pipeline collapsed the prompt into a generic marketplace CRUD schema.

RetainHQ Prompt

Result: Partial

Prompt included:

- Spaced repetition
- Active recall
- Review scheduling
- Slack
- Gmail
- Notion

Issues:

- Review schedule lost
- Active recall logic lost
- Recall ratings lost
- Business rules lost
- Workflows synthesized

Major entities survived but core product mechanics disappeared.

What We Will Not Work On

The following are intentionally out of scope:

- Animations
- New gradients
- Glassmorphism
- Marketing polish
- Landing pages
- Color palette changes
- Visual redesigns

These do not improve reliability, trust, or evaluation scores.

P0 — Must Ship

1. Overview Header

Render before all generated content.

Example:

Type: CRM

Entities: 2

Pages: 5

APIs: 10

Integrations: 1/1

Repairs: 0

Purpose:

A reviewer should understand the generated application within seconds.

Acceptance:

Visible at the top of every generation.

2. Prompt Understanding Panel

Render Stage 1 output.

Show:

- Detected Features
- Detected Entities
- Detected Integrations
- Assumptions

Example:

Detected Features

✓ Leads

✓ Properties

✓ Deals

✓ Analytics

Detected Integrations

✓ WhatsApp

Assumptions

- Admin manages all agents
- Deals follow a standard pipeline

Purpose:

Makes Intent Extraction visible.

Provides debugging visibility when information is lost later.

Acceptance:

Every generation shows parsed intent.

3. Business Rule Extraction

Extend AppIntent.

Current:

features[] entities[] integrations[]

Add:

businessRules[]

Examples:

- "Send WhatsApp when a deal closes"
- "Schedule reviews at +3/+7/+14/+30 days"
- "Create Notion page on topic mastery"
- "Send daily digest email"

Purpose:

Business rules are the missing link between intent and implementation.

Many downstream improvements depend on this data.

Acceptance:

Business rules extracted during Intent stage.

4. Repair Impact / Generation Health

Current:

Repair Log (14)

Problem:

Users cannot distinguish:

- Successful recovery
- Significant degradation

Render:

Generation Health

Healthy Repairs

✓ Missing array recovered

✓ tenantId injected

✓ inverse relation generated

Concerning Repairs

⚠ Workflow synthesized

⚠ Missing entity removed

⚠ Missing integration generated

⚠ Missing page generated

Purpose:

Not all repairs are equal.

Healthy repairs indicate resilience.

Concerning repairs indicate degraded output quality.

Acceptance:

Repairs categorized into Healthy vs Concerning groups.

5. Requirement Coverage

This becomes the primary trust feature.

Compare:

Stage 1 understanding

vs

Final AppSpec

Example:

Prompt Requirements

✓ Products

✓ Orders

✓ Customers

✓ Stripe

✓ Gmail

⚠ Payment entity missing

⚠ Order confirmation workflow missing

RetainHQ Example

✓ Study Activities

✓ Reviews

✓ Roadmaps

✓ Slack

✓ Gmail

✓ Notion

⚠ Recall Rating missing

⚠ Review Schedule missing

Purpose:

Show exactly what survived the pipeline.

Acceptance:

Every generation displays requirement coverage.

6. Workflow Generation Improvements

6.1 Prompt Hardening

Strengthen Stage 2 and Stage 3 prompts.

Requirements:

- Explicitly model transactional entities when mentioned
- Explicitly generate workflow stubs for integration-related requirements
- Explicitly generate dashboard/analytics surfaces when requested
- Avoid pure CRUD generation

Examples:

Payments → Payment entity

Analytics → Dashboard page

Chat → Chat entity/workflow

6.2 Rule-Aware Workflow Fallback

Current fallback:

entity[0]

status_changed

action[0]

Produces:

Product.status_changed

→ Stripe.create_customer

Fix:

Use businessRules when synthesizing workflows.

Acceptance:

Synthesized workflows derive triggers from business rules when available.

6.3 Workflow Labeling

Generated workflows should indicate origin.

Example:

Workflow

Deal Closed Notification

Source: LLM Generated

Workflow

Daily Review Digest

Source: Synthesized Fallback

Purpose:

Reviewers should know whether a workflow was inferred or synthesized.

Acceptance:

All synthesized workflows are clearly labeled.

P1 — High Value

7. Business Rule Coverage

Subset of Requirement Coverage.

Only displayed when businessRules exist.

Example:

Business Rules

✓ Slack when streak broken

✓ Notion on topic mastery

⚠ Review schedule (+3/+7/+14/+30)

⚠ Easy / Medium / Hard recall rating

Purpose:

Expose missing domain logic.

Acceptance:

Coverage computed against extracted business rules.

8. Integration Transparency

Current UI:

IMPLEMENTED

Problem:

This is inaccurate.

Reality:

- Registry exists
- Validation exists
- OAuth does not exist
- Live API calls do not exist

Replace:

IMPLEMENTED

with:

REGISTERED

or

SUPPORTED

or

SCHEMA READY

Add capability matrix:

Slack

Registry ✓

Validation ✓

OAuth ✗

Live API ✗

Purpose:

Avoid overstating implementation status.

Acceptance:

No integration is labeled implemented unless it truly is.

P2 — Nice To Have

9. Entity Purpose Descriptions

Example:

ReviewSession

Purpose:

Stores scheduled review events and recall results.

Helps users understand intent, not just schema.

10. Collapsible API Sections

Group APIs by entity.

Reduces noise for large applications.

11. ER Diagram

Visual representation of entity relationships.

Useful but not necessary for the trial submission.

Future Work

Deterministic Confidence Score

Do not build yet.

Confidence must be derived from:

- Requirement Coverage
- Business Rule Coverage
- Repair Health
- Workflow Quality
- Validation Success

Never:

- Ask the model for confidence
- Use subjective scoring
- Use self-reported confidence

Only build after the underlying metrics exist.

Final Priority Order

P0

1. Overview Header
2. Prompt Understanding
3. Business Rule Extraction
4. Repair Impact / Generation Health
5. Requirement Coverage
6. Workflow Generation Improvements

P1

1. Business Rule Coverage
2. Integration Transparency

P2

1. Entity Purpose Descriptions
2. Collapsible API Sections
3. ER Diagram

Future

1. Deterministic Confidence Score
-

Success Criteria

Re-run:

- CRM
- E-commerce
- Marketplace
- RetainHQ

The system must clearly show:

- ✓ What it understood
- ✓ What it assumed
- ✓ What it generated
- ✓ What it repaired
- ✓ What it missed
- ✓ Which workflows were synthesized
- ✓ Which integrations are merely registered
- ✓ Why the output should or should not be trusted

The submission should demonstrate not only generation capability, but also observability, reliability, and transparency.

That is more aligned with the evaluation criteria than any UI redesign.